



Universidad de Oviedo
Grado en Física

Introducción a la Física Computacional

Curso 2023–24

Módulos de cálculo científico en Python: Numpy

Índice general

7. Módulo <code>numpy</code>	3
7.1. Ventajas de usar <code>numpy</code>	5
7.2. La estructura básica de datos	6
7.3. Usando <code>numpy</code>	7
7.4. Creación de arrays	7
7.4.1. Creación de arrays a partir de una lista	7
7.4.2. Creación de arrays usando una función que retorna un array con valores	8
7.4.3. Leyendo datos de un fichero	14
7.5. Obteniendo información sobre arrays	15
7.5.1. Arrays con 'vistas'	15
7.5.2. Dimensiones de un array	16
7.5.3. Tamaño de un array	16
7.5.4. Forma de un array	16
7.5.5. La función <code>len</code>	16
7.5.6. Cambiando la forma	17
7.5.7. Añadiendo dimensiones	17
7.5.8. La traspuesta de un array	18
7.5.9. Arrays 'planos'	19
7.6. Acceso a elementos y secciones de un array	19
7.6.1. Acceso simple a los elementos de un array	20

7.6.2. Secciones de un array	20
7.6.3. Arrays conformables	21
7.6.4. Copiando secciones	21
7.6.5. Sub-arrays mediante listas	22
7.7. Operaciones con arrays	22
7.7.1. Suma de arrays	22
7.7.2. Resta de arrays	23
7.7.3. Multiplicación de arrays	23
7.7.4. División de arrays	24
7.7.5. Potenciación de arrays	24
7.7.6. Broadcasting	24
7.7.7. Comparación de arrays	29
7.7.8. La función <code>where</code>	29
7.8. Funciones universales	30
7.8.1. Definición de funciones que tienen como argumentos arrays	30
7.8.2. Vectorizando funciones	31
7.9. Funciones que devuelven valores	32
7.10. Ejercicios	37
7.11. Interés de vectorizar el código	38

Práctica 7

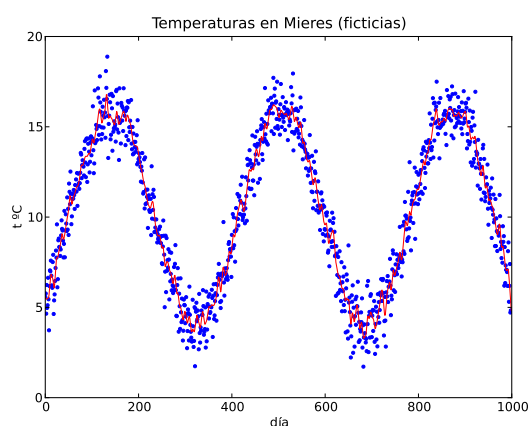
Módulo `numpy`

Este módulo proporciona una manera conveniente y eficiente de manejar vectores numéricos (los denominaremos *arrays* a partir de ahora) en Python. Los arrays son **homogéneos** (a diferencia de las listas de Python que pueden ser heterogéneas), los elementos de un array pueden ser de tipo booleano, entero, real o complejo (pero sin mezclarse). Un array puede tener varias dimensiones, a las que se accede por índices (ordenados). Además `numpy` dispone de una serie de rutinas que permiten operaciones rápidas sobre arrays. Con el fin de entender la potencia y la utilidad del uso de `numpy` plantearemos el siguiente ejemplo:

Ejemplo 7.0.1. *En el tratamiento de series de datos (temperatura, campo magnético, etc.) se utiliza a veces la media móvil, es decir, sustituir un elemento de la serie por su media con puntos anteriores y posteriores.*

Por ejemplo, si x es un vector cuyos índices varían de 0 a $n - 1$, tendríamos:

$$\begin{aligned}y_0 &= x_0, & y_{n-1} &= x_{n-1}, \\y_i &= \frac{1}{3}(x_{i-1} + x_i + x_{i+1}), & i &= 2, \dots, n-2\end{aligned}$$



Si los datos de temperatura los tenemos en el fichero `np01.txt` cuya estructura es de la forma:

```
# Temperaturas en Mieres (ficticias)
# Día   °C
0 5.5839
1 5.76949
2 4.65202
...
```

Una posible solución usando listas

```
'''
Ejemplo de programa en python para leer una serie de datos y calcular
la media móvil
'''

def mediamovil (x):
    ''' Calcula una media móvil de 3 puntos '''
    n = len(x)
    y = []
    for i, x1 in enumerate(x):
        if i == 0 or i == n-1:
            y.append(x1)
        else:
            y.append((x[i-1]+x1+x[i+1])/3)
    return y

# Lectura de datos
lineas = open("np01.txt").readlines()
x = []
for linea in lineas:
    if linea[0] == "#": continue
    columnas= linea.split()
    x.append(float(columnas[1]))

# Hacemos la media móvil
y = mediamovil(x)

# Hacemos cuatro medias móviles más
for i in range(4):
    y = mediamovil(y)
```

Una posible solución usando arrays de `numpy`

```
'''
Ejemplo de programa en python para leer una serie de datos y calcular
la media móvil usando numpy
'''
from numpy import *                                # <=== NUMPY

def mediamovil (x):
    ''' Calcula una media móvil de 3 puntos '''
    y = copy(x)                                    # <=== NUMPY
    y[1:-1] = (x[0:-2]+x[1:-1]+x[2:])/3           # <=== NUMPY
    return y

# Lectura de datos
nx = loadtxt("np01.txt")                          # <=== NUMPY
n, x = nx[:,0], nx[:,1]                          # <=== NUMPY

# Hacemos la media móvil
y = mediamovil(x)

# Hacemos cuatro medias móviles más
for i in range(4):
    y = mediamovil(y)
```

7.1. Ventajas de usar `numpy`

En los lenguajes interpretados (como `Python`) los bucles son lentos. Para realizar cálculo numérico es mejor que los bucles estén ‘escondidos’ en código compilado (que es mucho más rápido). Así pues, cualquier mecanismo que nos evite los bucles será muy bienvenido.

La eliminación de bucles no es especialmente necesaria en lenguajes compilados como C o Fortran. No obstante, con la llegada de ordenadores con varios procesadores muchas operaciones con arrays se pueden efectuar en **paralelo** en varios de ellos: idealmente cada procesador se encargaría de obtener la solución para un elemento del array.

`Numpy` nos permite eliminar bucles y eso también nos ahorra líneas de código (por tanto es más fácil de leer) y el código se parece más a la notación matemática. Además, si las rutinas de `numpy` en lenguaje compilado están desarrolladas para cálculo en paralelo también nos aprovechamos de los múltiples procesadores que hubiera en el ordenador.

También, y no menos importante, nos ‘obliga’ a *pensar en paralelo*: las operaciones son las mismas sobre diferentes elementos de los vectores. Los programas codificados con lenguajes ‘paralelos’ son normalmente más cortos, más claros y más cercanos a las ideas científicas que queremos programar.

Adaptar el código (cuando es posible) para que funcione en paralelo se denomina **vectorializar** o **vectorizar**. En general consiste en eliminar bucles con contadores. Más adelante, cuando conozcamos la sintaxis de `numpy` y sus posibilidades, veremos ejemplos de cómo vectorizar código.

7.2. La estructura básica de datos

En `numpy` el array es la estructura básica. Consiste realmente en un conjunto homogéneo de datos: `int`, `float` (por defecto), etc., que se disponen de manera adyacente en memoria en forma *unidimensional*. Si se requiere que el array tenga dos o más dimensiones, `numpy` nos proporciona (de manera transparente) la posibilidad de acceder a los elementos por sus índices. El array de `numpy` se denomina `ndarray` debido a que puede tener *n* dimensiones, por lo que el **eje** y las **dimensiones** son conceptos fundamentales en esta estructura de datos. El **eje** sirve para etiquetar cada **dimensión**, por ejemplo, un array 2D (2 dimensiones) tiene 2 ejes y un array 4D (4 dimensiones) tendrá 4 ejes. El número de **dimensiones** y elementos de un `ndarray` se define por su 'forma', que es una tupla de números enteros no negativos que especifican los tamaños de cada dimensión.

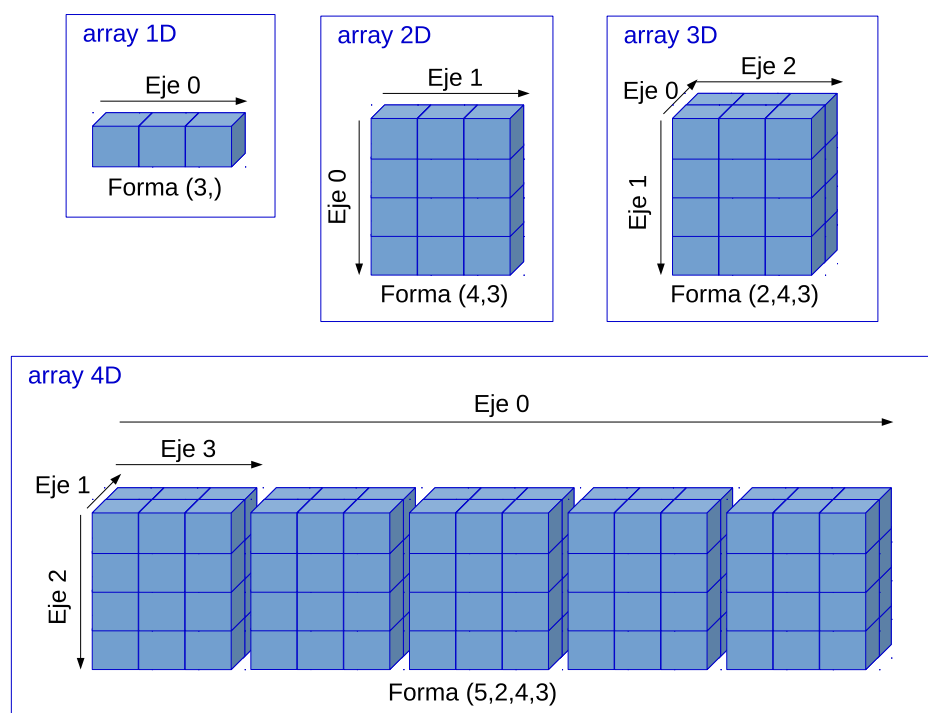


Figura 7.1: Visualización de un array multidimensional.

También existe la posibilidad de usar la estructura `matrix`, aunque creemos más recomendable usar `array` en su lugar (sobre todo en los comienzos del aprendizaje de `numpy`) pues su uso es más homogéneo. Existen muy pocas diferencias entre ambas, la principal es la definición del producto de

dos matrices (pero veremos que es muy sencillo con `array`).

7.3. Usando `numpy`

Para utilizar `numpy` lo primero que se debe hacer es cargar el módulo de la siguiente manera (por ejemplo):

```
from numpy import *
```

aunque también se puede usar el módulo con nombre:

```
import numpy as np
```

pero entonces en la invocación de las funciones tenemos que añadir delante `np`.

La mayoría de las funciones asociadas a `numpy` que vamos a utilizar a lo largo de este curso poseen más argumentos opcionales que los indicados. Para más información debe consultarse: <https://numpy.org/doc/stable/reference/>

El manejo de `arrays` conlleva diversos mecanismos:

- Creación de arrays.
- Obtención de información sobre los arrays.
- Acceso a elementos y secciones de arrays.
- Operaciones con arrays.
- Álgebra lineal y arrays aleatorios.

7.4. Creación de arrays

Existen tres maneras de crear arrays en `numpy`

- a partir de una lista o tupla (homogénea),
- usando una función que retorna un array con valores,
- leyendo datos desde un fichero.

7.4.1. Creación de arrays a partir de una lista

La función `array()` nos permite crear un array a partir de una lista.

Ejemplo 7.4.1. *Uso de la función `array()`.*

En el modo interactivo de Python escriba el siguiente código:

```
>>> from numpy import *
```

```
>>> a = array([3, 4, 2]) # 1D
>>> b = array([[3, 2, 4], [3, 3, 2]]) # 2D
>>> print(a)
[3 4 2]
>>> print(b)
[[3 2 4]
 [3 3 2]]
```

La función (constructor) `array()` genera un nuevo array. Por ejemplo si se utiliza `a = array(b)` siendo `b` un array entonces `a` es un nuevo array.

Si no queremos que haga la copia cuando `b` es un array se usará `asarray()`, de tal manera que `c = asarray(b)` genera un alias `c` para el array `b`. Si este último fuera una lista entonces `c` es un nuevo array.

Ejemplo 7.4.2. Uso de la función `asarray()`.

En el modo interactivo de Python escriba el siguiente código:

```
>>> a = array([3, 4, 2])
>>> b = array(a)
>>> print(a is b)
False
>>> c = asarray(a)
>>> print(a is c)
True
```

La operación inversa (convertir arrays en listas) se realiza con el método `tolist()` (atención, que `tolist(a)` no funciona).

Ejemplo 7.4.3. Uso de la función `tolist()`.

En el modo interactivo de Python escriba el siguiente código:

```
>>> a = array([[3., 2., 4.], [3., 3., 2.]])
>>> a = a.tolist()
>>> print(a)
[[3.0, 2.0, 4.0], [3.0, 3.0, 2.0]]
>>> b = array([2., 0., -1.])
>>> b = b.tolist()
>>> print(b)
[2.0, 0.0, -1.0]
```

7.4.2. Creación de arrays usando una función que retorna un array con valores

Algunas de las funciones que crean arrays son:

`arange`, `linspace`, `logspace`, `zeros`, `ones`, `zeros_like`, `ones_like`, `eye`, `copy`, `concatenate`, `repeat`, `mgrid` y `ogrid`.

La función `arange()`

Es similar a `range`, pero en vez de crear un objeto iterable crea un array y además se puede usar un espaciado de tipo `float`.

```
>>> a = arange(5)
>>> print(a)
[0 1 2 3 4]
>>> c = arange(1, 2, 0.2)
>>> print(c)
[ 1.  1.2  1.4  1.6  1.8]
```

El parámetro `0.2` representa el espaciado entre los puntos 1 y 2, donde 2 está excluido.

La función `linspace()`

Esta función produce un array con un número determinado de elementos linealmente espaciados entre dos límites:

```
>>> b = linspace(1, 2, 6)
>>> print(b)
[1.  1.2  1.4  1.6  1.8  2.]
```

Se incluyen 6 puntos desde 1 hasta 2 y a diferencia de `arange`, se incluyen los extremos.

La función `logspace()`

Produce un array con un número determinado de elementos logarítmicamente espaciados entre dos límites:

```
>>> b = logspace(1, 2, 5)
>>> print(b)
[ 10.  17.7827941  31.6227766  56.23413252 100. ]
```

Se incluyen 5 puntos desde 10^1 hasta 10^2 . Como en el caso de `linspace()`, se incluyen los extremos.

La función `ones()`

Crea un array con todos unos con el número de elementos indicados:

```
>>> a = ones(3)
>>> print(a)
[1.  1.  1.]
```

Si se pasa como argumento una tupla usa ese número de elementos en cada dimensión:

```
>>> b = ones((3, 2)) # 3 filas, 2 columnas
>>> print(b)
[[1.  1.]
```

```
[1. 1.]  
[1. 1.]
```

La función `zeros()`

Crea un array con todo ceros con el número de elementos indicados:

```
>>> b = zeros((3, 2)) # 3 filas, 2 columnas  
>>> print(b)  
[[0. 0.]  
 [0. 0.]  
 [0. 0.]
```

Para más de una dimensión el primer argumento de `ones` y `zeros` debe ser una tupla (de ahí los dobles paréntesis).

Las funciones `zeros_like()` y `ones_like()`

Crea un array con todo ceros o unos con la misma forma que otro array:

```
>>> a = array([[1, 2, 3, 4], [5, 6, 7, 8]])  
>>> ceros = zeros_like(a) # 2 filas, 4 columnas  
>>> print(ceros)  
[[0 0 0 0]  
 [0 0 0 0]]  
>>> unos = ones_like(a) # 2 filas, 4 columnas  
>>> print(unos)  
[[1 1 1 1]  
 [1 1 1 1]]
```

La función `eye()`

Crea un array con la diagonal principal llena de unos y el resto de los elementos ceros:

```
>>> b = eye(4, 3) # 4 filas, 3 columnas  
>>> print(b)  
[[1. 0. 0.]  
 [0. 1. 0.]  
 [0. 0. 1.]  
 [0. 0. 0.]
```

Como vemos, no hace falta que el array sea cuadrado.

También se puede indicar que los unos estén por encima o debajo de la diagonal principal usando un parámetro `k` (0, por defecto, correspondiente a la diagonal principal) y puede tomar valores positivos (por encima de la diagonal principal) y negativos (por debajo):

```
>>> b = eye(5, 4, k = -1)
>>> print(b)
[[ 0.  0.  0.  0.]
 [ 1.  0.  0.  0.]
 [ 0.  1.  0.  0.]
 [ 0.  0.  1.  0.]
 [ 0.  0.  0.  1.]]
```

La función `copy()`

Cuando se asigna una variable a un array preexistente no se copia el array sino que se crea otro nombre para el array:

```
>>> x = array([1, 2, 3])
>>> y = x # y es una 'vista' de x
>>> x[0] = 10 # luego veremos esta orden
>>> print(x)
[10 2 3]
>>> print(y)
[10 2 3]
```

Realmente se puede considerar a `y` (también a `x`) como una **ventana** que mira en el verdadero array (un objeto en memoria con características de array).

La función `copy()` realiza una copia a partir de otro array. También existe como método. En su versión más simple (y usual) sería:

```
>>> x = array([1, 2, 3])
>>> y = copy(x) # Copia profunda (deep copy). También y = x.copy()
>>> x[0] = 10
>>> print(x)
[10 2 3]
>>> print(y)
[1 2 3]
```

El nuevo array al que referencia `y` no es el mismo que el antiguo `x`.

La función `concatenate()`

La función `concatenate()` sirve para concatenar varios arrays y tiene el siguiente formato:

```
concatenate((a1, a2, ...), axis=0)
```

El primer argumento es una tupla con los arrays que se quieren concatenar. El argumento `axis` (por defecto 0) permite definir en qué eje (dimensión) se quiere realizar la concatenación. Este eje, se corresponde con el índice de la tupla que identifica la forma del array. La forma de los arrays ha de ser la misma excepto en la dimensión correspondiente a `axis`.

```
>>> a = array([[1, 2, 3], [4, 5, 6]]) # Forma (2,3)
```

```

>>> b = array([[7, 8, 9]])# Forma (1,3)
>>> concatenate((a, b))
>>> # (OJO con los paréntesis)
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
>>> a = array([[1, 2], [4, 5], [5, 6]]) # Forma (3,2)
>>> b = array([[7],[8],[9]])# Forma (3,1)
>>> concatenate((a, b)) # Da error
Traceback (most recent call last):
  File <stdin>", line 1, in <module>
  File <__array_function__ internals>", line 5, in concatenate
ValueError: all the input array dimensions for the concatenation axis
must match exactly, but along dimension 1, the array at index 0 has size
2 and the array at index 1 has size 1
>>> concatenate((a, b), axis=1) # A lo largo de las columnas
array([[1, 2, 7],
       [3, 4, 8],
       [5, 6, 9]])

```

Los arrays 1D concatenan generando un array 1D:

```

>>> x = array([0, 1, 2])
>>> y = array([-1, 0.2, 0.9])
>>> concatenate((x, y))
array([ 0. , 1. , 2. , -1. , 0.2, 0.9])

```

Si queremos concatenarlos generando varias filas se debe usar el siguiente 'truco':

```

>>> concatenate(([x], [y])) # fíjemonos en los corchetes rojos
array([[ 0. , 1. , 2. ],
       [-1. , 0.2, 0.9]])

```

La función `repeat()`

La función `repeat(a, copias, axis)` sirve para extender arrays a lo largo de una dimensión. También se puede poner `a.repeat(copias, axis)`. En su modo más simple de funcionamiento conviene indicar la dimensión ('axis') en la que queremos repetir el array:

```

>>> a = array([[1, 2], [3, 4]])
>>> repeat(a, 2, axis=0)
array([[1, 2],
       [1, 2],
       [3, 4],
       [3, 4]])

>>> repeat(a, 2, axis=1)
array([[1, 1, 2, 2],
       [3, 3, 4, 4]])

```

Si no indicamos el eje:

```
>>> repeat(a, 2)
array([1, 1, 2, 2, 3, 3, 4, 4])
```

La “función” `ogrid`

Esta “función” (en realidad es una clase) se utiliza para generar arrays usados en **mallados**. Su sintaxis es la siguiente:

`m = ogrid[rango_x, rango_y]`, siendo cada rango:

`rango  $\equiv$  inicio_incluido : fin_excluido : paso`

Nos devuelve una lista con dos arrays 2D: el primero contiene un mallado en el eje x y el segundo un mallado en el eje y (figura 7.2).

```
>>> p = ogrid[0:2:0.5, 0:3] # Atención: Se usa con corchetes
>>> print(p)
[array([[ 0. ],
        [ 0.5],
        [ 1. ],
        [ 1.5]]), array([[ 0., 1., 2.]])]
```

Observe que la forma de cada array es diferente, para x el array es de la forma $(nx,1)$ siendo nx el número de puntos del mallado en x y para y el array es de la forma $(1,ny)$ con ny el número de puntos para el mallado en y . Combinando ambos arrays se obtiene un mallado bidimensional. El mismo efecto se puede conseguir con `linspace()` aplicado a cada uno de los ejes y cambiando su forma de manera adecuada.

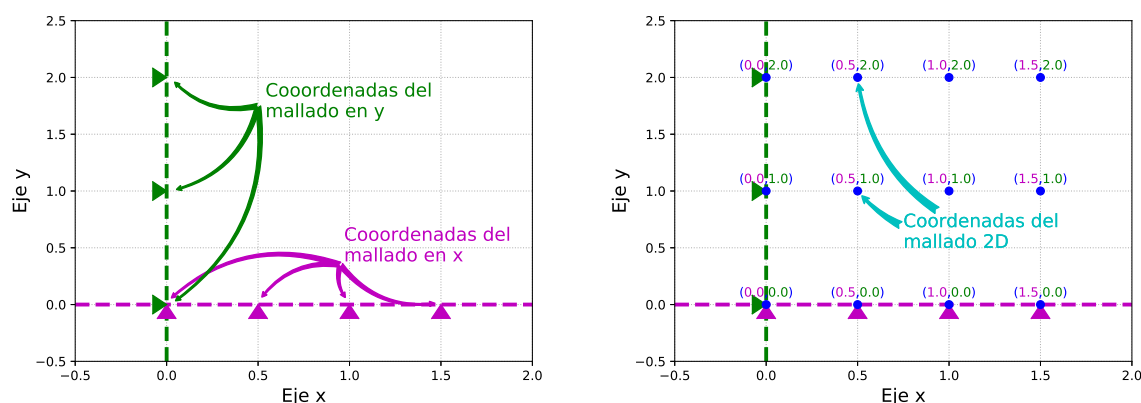


Figura 7.2: Visualización de los mallados de `ogrid` y de `mgrid`

La “función” `mgrid`

Se utiliza también para generar arrays usados en mallados. Su sintaxis es la siguiente:

`m = mgrid[rango_x, rango_y]`, siendo cada rango:

`rango  $\equiv$  inicio_incluyente : fin_excluyente : paso`

Nos devuelve un array 3D que, en realidad, consta de 2 arrays 2D, X (`m[0]`) e Y (`m[1]`). La combinación elemento a elemento de X e Y produce las coordenadas (x_{ij}, y_{ij}) de una malla en el plano (figura 7.2).

```
>>> m = mgrid[0:2:0.5, 0:3] # Atención: Se usa con corchetes
>>> print(m)
[[ 0.  0.  0. ]
 [ 0.5 0.5 0.5]
 [ 1.  1.  1. ]
 [ 1.5 1.5 1.5]]

[[ 0.  1.  2. ]
 [ 0.  1.  2. ]
 [ 0.  1.  2. ]
 [ 0.  1.  2. ]]
```

7.4.3. Leyendo datos de un fichero

Existen varias funciones para leer datos (preparados previamente de cierta manera) a partir de ficheros. La función básica es `loadtxt()`. Su sintaxis es la siguiente:

```
out = loadtxt(fname, dtype='float', comments='#',
              delimiter=None, skiprows=0, usecols=None)
```

Este ejemplo lee del fichero de texto cuyo nombre está guardado en la variable `fname` y los incluye en el array `out`. Sólo se exponen algunos argumentos que tienen puestos los valores por defecto:

`dtype` indica el tipo de dato (entero, real, etc.), por defecto `float`.

`comments` indica un string por el que empiezan las líneas con comentarios (que se saltan entonces). Por defecto es `#`.

`delimiter` indica los caracteres que se usan para separar datos, por defecto espacios.

`skiprows` indica cuántas filas iniciales se saltan antes de empezar a leer realmente el array (por defecto 0). Las líneas de comentarios se cuentan también.

`usecols` indica una secuencia de columnas para leer (por defecto todas), por ejemplo `usecols=(1,3,5)` lee sólo esas columnas.

Ejemplo 7.4.4. *Uso de la función `loadtxt`. Escriba el siguiente código en un fichero con el nombre `z.txt`:*

```
# z.txt -----
1 2 3
# otro comentario
4 5 6
7 8 9
#-----
```

Una vez guardado el archivo escriba en el modo interactivo de Python:

```
>>> out=loadtxt('z.txt', skiprows=2, usecols=(1,2), dtype='int')
>>> print(out)
[[5 6]
 [8 9]]
>>> b = loadtxt('z.txt')
>>> print(b)
[[ 1.  2.  3.]
 [ 4.  5.  6.]
 [ 7.  8.  9.]]
```

7.5. Obteniendo información sobre arrays

Recordemos que los arrays tienen asociados un número de dimensiones y un tamaño a lo largo de cada dimension definido en su 'forma' (ver figura 7.1). Se puede acceder a esa información de diversas maneras, usando `ndim`, `size`, `shape`, `len` y `reshape`.

7.5.1. Arrays con 'vistas'

Los datos del array (o sea los valores de sus elementos) son los mismos independientemente de la forma asociada (puesto que los arrays se guardan de manera secuencial en memoria).

Al cambiar la forma de un array se modifica la 'vista' (o sea, cómo se ve el array).

Situación real de `a` en memoria:

1	2	3	4	5	6
---	---	---	---	---	---

Vista con forma `(2,3)`:

1	2	3
4	5	6

Vista con forma `(3,2)`:

1	2
3	4
5	6

Todas estas vistas son el mismo array.

7.5.2. Dimensiones de un array

Para determinar el número de dimensiones se utiliza `ndim`:

```
>>> a = array([[3, 2, 4], [3, 3, 2]])
>>> ndim(a)
2
>>> a.ndim
2
```

Como vemos `a.ndim` es un dato entero no una función.

7.5.3. Tamaño de un array

Para determinar el número total de elementos (tamaño) se utiliza `size`:

```
>>> a = array([[3, 2, 4], [3, 3, 2]])
>>> size(a)
6
>>> a.size
6
```

Como vemos `a.size` es un dato entero no una función.

7.5.4. Forma de un array

Para determinar la forma, o sea el número de elementos en cada dimensión se utiliza `shape`:

```
>>> a = array([[3, 2, 4], [3, 3, 2]])
>>> shape(a)
(2, 3)
>>> a.shape
(2, 3)
```

Como vemos `a.shape` es una tupla de enteros no una función.

7.5.5. La función `len`

Esta función aplicada a un array nos da el tamaño correspondiente a la primera dimensión:

```
>>> a = array([[3, 2, 4], [3, 3, 2]])
>>> a.shape[0]
2
>>> len(a)
2
```

igual que se obtendría aplicada a una lista de listas.

7.5.6. Cambiando la forma

Se puede cambiar la forma de un array con la función `reshape` o asignando una nueva tupla a la variable `a.shape`:

```
>>> a = array([[3, 2, 4], [3, 3, 2]])
>>> a.shape
(2, 3)
>>> a.shape = (a.size, ) # es equivalente a.shape = (-1, )
>>> print(a)
[3 2 4 3 3 2]
```

Hemos cambiado las dimensiones del array, aunque no su tamaño (o sea 6 en este caso): `a.size` no puede cambiar.

En el ejemplo anterior, a partir de este momento el array tiene una sola dimensión y se accede (como veremos posteriormente) con un solo índice:

```
>>> a[0] = 100
>>> a.shape = (2, 3)
>>> print(a)
[[100 2 4]
 [ 3 3 2]]
```

Vemos que las modificaciones realizadas en `a` se transmiten aunque se cambie de nuevo la forma.

También podemos cambiar la forma recurriendo a la función `reshape`, en este caso el procedimiento será el siguiente:

```
>>> a = array([[1, 2, 3], [4, 5, 6]])
>>> print(a)
[[1, 2, 3]
 [4, 5, 6]]
>>> a=a.reshape(3, 2)
>>> print(a)
[[1, 2]
 [3, 4]
 [5, 6]]
```

7.5.7. Añadiendo dimensiones

Se pueden añadir dimensiones (con un número de elementos 1, obviamente) mediante dos funciones. La primera manera se utiliza para añadir dimensiones al principio o al final de un array ya conocido.

Si `a` es un array, `a[newaxis]`, añade un eje al principio:

```
>>> b = array([[1, 2, 3], [4, 5, 6]])
>>> b.shape
(2, 3)
>>> c = b[newaxis]
```

```
>>> c.shape
(1, 2, 3)
```

`a[newaxis]` es equivalente a `a[newaxis,...]`. Por otro lado `a[...,newaxis]` añade la dimensión al final:

```
>>> d = b[newaxis,...]
>>> d.shape
(2, 3, 1)
```

Como vemos los puntos suspensivos indican todas las dimensiones que tiene el array original.

Se puede añadir una dimensión en una posición intermedia con la función `expand_dims(a, axis=axis)`, donde el argumento `axis` es obligatorio:

```
>>> g = expand_dims(b, axis=1)
>>> g.shape
(2, 1, 3)
```

Una cosa importante. No se puede poner `a.expand_dims(axis=axis)`, sino que debe invocarse como está indicado más arriba.

7.5.8. La traspuesta de un array

Si `a` es un array, `transpose(a)`, `a.transpose()` y `a.T` (que hacen lo mismo), nos devuelven una vista traspuesta del array `a`:

```
>>> b = array([[1, 2, 3], [4, 5, 6]])
>>> print(b.T)
[[1 4]
 [2 5]
 [3 6]]
```

`transpose` por defecto invierte todas las dimensiones.

Se puede indicar el orden exacto de las dimensiones mediante una lista opcional de ejes, para comprobarlo escriba el siguiente código:

```
>>> a = arange(1,25)
```

Hemos creado un array `a` 1D, al que cambiaremos a 3D de la siguiente manera:

```
>>> a.shape = (2,3,4)
```

Compruebe la nueva forma de `a` imprimiéndolo en pantalla.

Si en el array anterior queremos intercambiar filas y columnas debemos ejecutar:

```
>>> transpose(a, axes=(0,2,1))
```

Ejecute a continuación `a.T` y compruebe la diferencia con la transpuesta anterior.

Esto se puede realizar con arrays de mas dimensiones. Creemos un nuevo array `a` de la forma (2,3,4,5), para lo cual escribiremos el siguiente código:

```
>>> a = linspace(1,120,120)
>>> a.shape=(2,3,4,5)
```

Compruebe la nueva forma de `a` imprimiéndolo en pantalla.

Si en este nuevo array queremos intercambiar filas y columnas debemos ejecutar:

```
>>> transpose(a, axes=(0,1,3,2))
```

Vuelva a comprobar el resultado obtenido a partir de `a.T` con el anterior.

7.5.9. Arrays ‘planos’

La función `ravel` nos devuelve una **vista** de una única dimensión, mientras que `flatten` nos devuelve una **copia**:

```
>>> b = array([[3, 2, 4], [3, 3, 2]])
>>> print(b)
[[3 2 4]
 [3 3 2]]
>>> c = ravel(b) #es una vista de b
>>> print(c)
[3 2 4 3 3 2]
>>> d = b.flatten() #es una copia independiente
```

Compruebe que si modificamos `c` se modifica `b` y que si modificamos `d` no se modifica `b`.

7.6. Acceso a elementos y secciones de un array

Para acceder a partes de un array existen diversos mecanismos:

- acceso a un elemento concreto,
- vistas de secciones (*slices* en inglés),
- copia de secciones.

7.6.1. Acceso simple a los elementos de un array

Para acceder a los elementos de un array se usan índices enteros, en lectura o escritura (como a continuación):

```
>>> b = array([[3, 2, 4], [3, 3, 2]])
>>> b[1][1] = 10 # una manera
>>> b[0, 0] = 11 # otra manera más eficiente
>>> print(b)
[[11 2 4]
 [3 10 2]]
```

7.6.2. Secciones de un array

La potencia de los arrays se manifiesta en el manejo de secciones (*slices* en inglés) de los mismos.

Las secciones siempre proporcionan vistas, no hacen copia de los arrays.

Una sección es un bloque (formado por elementos a veces no contiguos) dentro de un array. El bloque se define a través de **patrones que verifican los índices** (tantos como dimensiones).

Supongamos un array, *a*, bidimensional por simplificar:

00	01	02	03	04
10	11	12	13	14
20	21	22	23	24
30	31	32	33	34

El primer índice indica la fila y el segundo la columna.

Patrón: *inicio:final:incremento*.

El inicio es inclusivo y el final excluyente.

Ejemplo 7.6.1. Acceso a secciones de un array.

Introdúzca el array *a* en una consola de Python.

Se quiere extraer las columnas impares. Para ello generamos un patrón de fila *:* (que indica todo el rango) y uno de columna *0::2*.

Se genera una vista de los elementos que están indicados en rojo y negrita:

00	01	02	03	04
10	11	12	13	14
20	21	22	23	24
30	31	32	33	34

Para extraerlo tendríamos el siguiente código:

a[:, 0::2] (y también *a[:, ::2]*)

que genera un array bidimensional:

00	02	04
10	12	14
20	22	24
30	32	34

Cualquier modificación en una sección del array `a` se manifiesta en cambios en el propio array `a`:

```
>>> a[:, 0::2] = 77 # Nota
>>> print(a)
[[77  1 77  3 77]
 [77 11 77 13 77]
 [77 21 77 23 77]
 [77 31 77 33 77]]
```

Nota: Esa asignación se denomina *broadcasting*, y es la conversión de un dato en el necesario en una operación (asignación a un array en nuestro caso).

Ejercicio 7.6.1. Acceso a secciones del array `a`.

1. ¿Qué devuelve `a[:, ::-1]`?
2. Obtenga las filas impares de `a`.
3. Obtenga la vista cuadrada eliminando la última columna.
4. Obtenga una vista con las dos primeras líneas.
5. Obtenga una vista con los elementos comunes de las líneas impares y las columnas impares.

7.6.3. Arrays conformables

Este array `a[:, 0::2]` concreto es **conformable** con cualquier array con forma `(4, 3)`.

Eso significa que puede sustituirse donde se necesite un array de esa forma. Por ejemplo se puede sumar a un array de forma `(4, 3)`.

Ejercicio 7.6.2. Introduzca en la terminal de Python un array de unos con forma `(4, 3)` y realice la suma entre el array de unos y el array `a[:, 0::2]`.

También se puede multiplicar matricialmente (luego lo veremos) con un array de forma `(3, ...)`.

7.6.4. Copiando secciones

Para generar una copia y no una vista se debe utilizar la función `copy` acompañada de la selección de sección correspondiente:

```
>>> b = copy(a[:, :2])
```

```
>>> print(b)
[[ 0  2  4]
 [10 12 14]]
>>> b[0,0] = 1000 # no modifica a[0,0]
```

7.6.5. Sub-arrays mediante listas

Se pueden utilizar también listas como patrones de selección de secciones de arrays. En este caso se obtiene una **copia** y no una vista:

```
>>> c = a[:2:, ::2] # sección
>>> b = a[:2:, [0,2,4]] # copia
# también b = a[:2:, range(0,6,2)]
>>> print(b)
[[ 0  2  4]
 [10 12 14]]
>>> b[0,0] = 1000 # no modifica a[0,0]
>>> c[0,0] = 2222
# sí modifica a[0,0]
```

7.7. Operaciones con arrays

Existen operaciones, algunas elemento a elemento (en inglés *elementwise*), y otras resultado de llamadas a funciones que retornan otros arrays o escalares.

Las operaciones básicas son: suma (+), resta (-), multiplicación (*), división (/), potenciación (**) y las comparaciones. También se aplican elemento a elemento las denominadas **funciones universales**.

Para poder operar entre sí dos arrays deben tener la misma forma. Realmente es algo más complicado, pero no lo describiremos aquí. En el argot del cálculo en paralelo se dice que los arrays deben ser *conformables*.

Las funciones que devuelven valores son: 'y' lógico ([all](#)), 'o' lógico ([any](#)), máximo ([max](#)), mínimo ([min](#)), suma ([sum](#)), producto ([prod](#)), media ([mean](#)), desviación estándar o típica ([std](#)), producto matricial ([dot](#)), producto vectorial ([cross](#)), producto externo ([outer](#)).

7.7.1. Suma de arrays

Para poder sumar dos arrays deben tener la misma forma:

```
>>> a = linspace(1,6,6)
>>> a.shape=(2,3)
>>> print(a)
[[1.  2.  3.]
```

```
[4. 5. 6.]
>>> b = linspace(100,600,6)
>>> b.shape(2,3)
>>> print(b)
[[100. 200. 300.]
 [400. 500. 600.]]
>>> c = a+b
>>> print(c)
[[101. 202. 303.]
 [404. 505. 606.]]
>>> print(a+array([1, 2, 3, 4])) # ERROR
La suma de arrays genera un nuevo array.
```

También se puede utilizar el operador `+=` que no crea un nuevo array sino que modifica el que está a la izquierda:

```
>>> a += b
>>> print(a)
[[101. 202. 303.]
 [404. 505. 606.]]
```

7.7.2. Resta de arrays

Para poder restar dos arrays deben tener la misma forma. Si al array `a` obtenido en la operación anterior le restamos el array `b`:

```
>>> c = a-b
>>> print(c)
[[1. 2. 3.]
 [4. 5. 6.]]
```

También se puede usar `-=` de manera similar a `+=`.

7.7.3. Multiplicación de arrays

Para poder multiplicar (elemento a elemento) dos arrays deben tener la misma forma:

```
>>> a = linspace(1,6,6)
>>> a.shape=(2,3)
>>> print(a)
[[1. 2. 3.]
 [4. 5. 6.]]
>>> c = a*b
>>> print(c)
[[100. 400. 900.]
 [1600. 2500. 3600.]]
```

También se puede usar `*=` de manera similar a `+=`.

7.7.4. División de arrays

Para poder dividir (elemento a elemento) dos arrays deben tener la misma forma:

```
>>> c = a/b
>>> print(c)
[[0.01 0.01 0.01]
 [0.01 0.01 0.01]]
```

También se puede usar `/=` de manera similar a `+=`.

7.7.5. Potenciación de arrays

La potencia también se aplica elemento a elemento y los dos arrays deben tener la misma forma:

```
>>> c = b**a # también c = pow(b, a)
>>> print(c)
[[1.0000e+02 4.0000e+04 2.7000e+07]
 [2.5600e+10 3.1250e+13 4.6656e+16]]
```

También se puede usar `**=` de manera similar a `+=`.

Ejercicio 7.7.1. Demuestre que $\lim_{n \rightarrow \infty} (1 + \frac{1}{n})^n = e$. Para hacer esto cree un vector n que tenga los elementos: 1, 10, 100, 500, 1000, 2000, 4000 y 8000. Seguidamente cree un vector y en el cual cada elemento será calculado a partir de los elementos de n mediante $(1 + \frac{1}{n})^n$.

Ejercicio 7.7.2. La profundidad de un pozo, h , en metros se puede determinar a partir del tiempo que tarda en caer una piedra en su interior (con velocidad inicial 0). Este cálculo viene determinado por: $h = \frac{1}{2}gt^2$, donde t es el tiempo en segundos y $g = 9.81 \text{ m/s}^2$. Calcule h para $t = 1, 2, 3, 4, 5, 6, 7, 8, 9$ y 10 s.

7.7.6. Broadcasting

Este término describe cómo trata `numpy` arrays de **diferentes formas** en las operaciones aritméticas.

La aplicación en casos generales de las reglas de *broadcasting* es relativamente compleja para el nivel de estos apuntes. Para más información, véase la documentación de `numpy` y también lo expuesto en <https://numpy.org/doc/stable/user/basics.broadcasting.html>.

Hemos separado las reglas para mejor entendimiento:

1. Un escalar siempre se puede operar con cualquier array.
2. Dos dimensiones son compatibles en las operaciones aritméticas cuando son iguales o cuando una de ellas es 1. Los tamaños de las dimensiones que son 1 se extienden hasta completar el tamaño del otro array.
3. Cuando el número de dimensiones de los arrays es diferente, se compara el tamaño de las dimensiones empezando por la derecha, y entonces los tamaños deben ser iguales, extendiéndose el array más corto hasta completar el tamaño del más largo.

4. Las reglas anteriores pueden combinarse.

Así pues, siempre que se pueda realizar *broadcasting* según las reglas anteriores, las dimensiones menos completas de cada array se ‘extienden’ para que los arrays al operar parezcan tener la misma forma. Todos los cálculos se hacen sin hacer realmente copias. Por tanto, el *broadcasting* es una operación computacionalmente muy eficiente.

Vamos a ver ejemplos de cada regla por separado:

Empecemos por el caso 1, con operaciones entre escalares y arrays.

Ejemplo 7.7.1. Operaciones entre escalares y arrays.

En el modo interactivo escriba el siguiente código:

```
>>> a = arange(1,4,0.5)
>>> a.shape = (2,3)
>>> print(a)
[[1.  1.5  2. ]
 [2.5  3.  3.5]]
>>> print(a+2)
[[3.  3.5  4. ]
 [4.5  5.  5.5]]
>>> print(a**2)
[[1.  2.25  4. ]
 [6.25  9.  12.25]]
```

El escalar 2 se extiende para formar un array de forma (2,3) antes de operar:

```
2 -> [[2  2  2]
      [2  2  2]]
```

Pasemos al caso 2, con dos arrays de la misma dimensión y de diferentes tamaños, por ejemplo:

```
a = [[1.  2.  3.]
      [4.  5.  6.]]
g = [[100]
      [200]]
```

Como vemos `a.shape = (2, 3)` y `g.shape = (2, 1)`, y por tanto estamos en el caso expuesto. El array `g` se puede ‘extender’ tres veces a lo largo de segunda dimensión, repitiéndose de la manera:

```
g -> [[100 100 100]
      [200 200 200]]
```

antes de sumarse al array `a`.

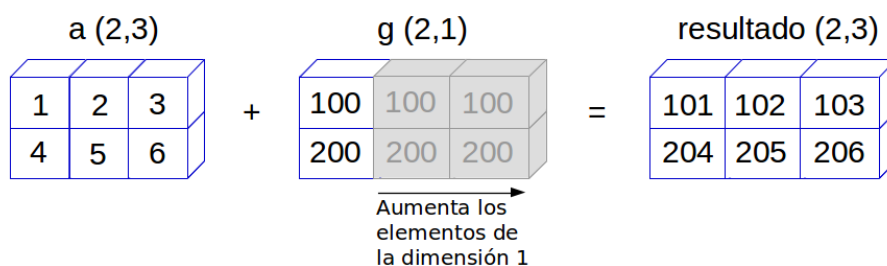


Figura 7.3: Visualización del ejemplo para el caso 2 de broadcasting.

Ejemplo 7.7.2. Operaciones entre arrays de la misma dimensión y diferente forma.

Escriba el siguiente código:

```
>>> a = arange(1,7)
>>> a.shape=(2,3)
>>> print(a)
[[ 1.  2.  3.]
 [ 4.  5.  6.]]
>>> g = array([[100],[200]]) # forma (2, 1)
>>> print(g)
[[ 100 ]
 [ 200 ]]
>>> print(a+g)
[[ 101. 102. 103.]
 [ 204. 205. 206.]]
```

El caso 3 hace referencia a la forma de operar con dos arrays de diferente dimensión. En ese caso se comparan las dimensiones a lo largo de cada eje (*axis*) empezando por la derecha. Tienen que coincidir hasta acabarse el número de dimensiones del array más 'corto'. Por ejemplo:

```
a = [[ 0  0  0]
      [10 10 10]
      [20 20 20]
      [30 30 30]]
b = [1 2 3]
```

Como vemos `a.shape = (4, 3)` y `b.shape = (3,)`, y por tanto estamos en el caso expuesto: el array `b` se puede 'extender' a lo largo de una nueva dimensión 0, repitiéndose de la manera:

```
b = [[1 2 3]
      [1 2 3]
      [1 2 3]
      [1 2 3]]
```

antes de sumarse al array `a`.

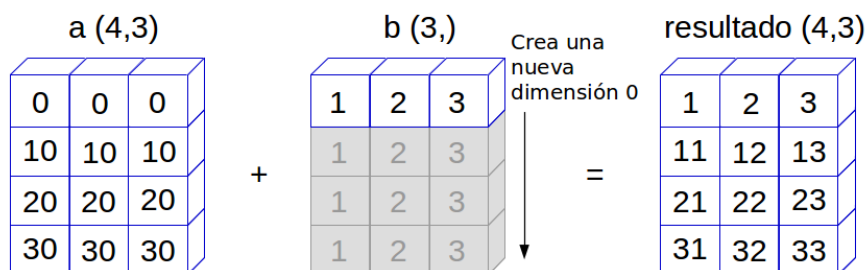


Figura 7.4: Visualización del ejemplo para el caso 3 de broadcasting.

Ejemplo 7.7.3. Operaciones entre arrays de diferente dimensión.

Escriba el siguiente código:

```
>>> a = arange(0,40,10)
>>> print(a)
[ 0. 10. 20. 30.]
>>> a.shape=(size(a),1)
>>> print(a)
[[ 0. ]
 [10. ]
 [20. ]
 [30. ]]
>>> a = repeat(a,3,axis=1)
>>> print(a) # array 2D con forma (4,3)
[[ 0.  0.  0.]
 [10. 10. 10.]
 [20. 20. 20.]
 [30. 30. 30.]]
>>> b = linspace(1,2,3)
>>> print(b) # array 1D con forma (3,)
[ 1.  2.  3.]
>>> print(a+b)
[[ 1.  2.  3.]
 [11. 12. 13.]
 [21. 22. 23.]
 [31. 32. 33.]]
```

La regla 4 hace referencia a que se pueden combinar todas las reglas anteriores, por ejemplo:

```
a = [[ 0]
      [10]
      [20]
      [30]]
b = [1 2 3]
```

Como vemos `a.shape = (4, 1)` y `b.shape = (3, )`. Ambos arrays se 'extienden', antes de sumarse, lo necesario, repitiéndose como se muestra en la figura siguiente:

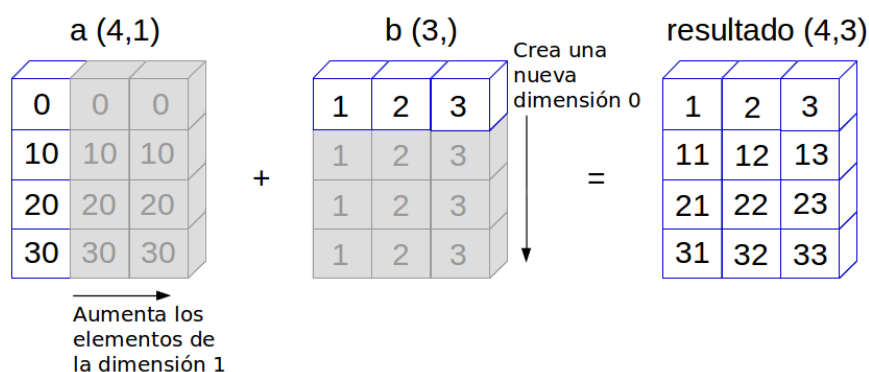


Figura 7.5: Visualización del ejemplo para el caso 4 de broadcasting.

Ejemplo 7.7.4. Operaciones entre arrays que combinan las reglas anteriores.

Escriba el siguiente código:

```
>>> a = arange(0,40,10)
>>> print(a)
[ 0. 10. 20. 30.]
>>> a.shape = (size(a), 1)
>>> print(a)
[[ 0. ]
 [10. ]
 [20. ]
 [30. ]]
>>> b=linspace(1,3,3)
>>> print(b)
[ 1.  2.  3.]
# a.shape = (4, 1), v.shape = (3,)
>>> print(a+b)
[[ 1.  2.  3.]
 [11. 12. 13.]
 [21. 22. 23.]
 [31. 32. 33.]
```

Otro caso con más dimensiones sería sumar los arrays `A` y `B` con la estructura siguiente:

```
A    (4d array): 8 x 1 x 6 x 1
B    (3d array):   7 x 1 x 5
A+B  (4d array): 8 x 7 x 6 x 5
```

Vemos cómo se extienden tanto `A` como `B` en las dimensiones de tamaño 1, y en la que falta al principio de `B`.

Ejercicio 7.7.3. Cree un array `a` de la forma (8,1,6,1) cuyos elementos sean números enteros y un array `b` de la forma (7,1,5) que contenga números en coma flotante y realice las siguientes operaciones aritméticas: `a+b`, `a-b` y `a/b`

7.7.7. Comparación de arrays

Las operaciones de comparación devuelven arrays con valores lógicos:

```
>>> a = linspace(0,5,6)
>>> a.shape = (2,3)
>>> print(a)
[[0. 1. 2.]
 [3. 4. 5.]]
>>> b = a > 2
>>> print(b)
[[ False False  False]
 [ True  True   True]]
```

No obstante, si los operadores de comparación se usan en una operación, `True` se convierte en `1` y `False` se convierte en `0`:

```
>>> c = 9*(a > 2)
>>> print(c)
[[0 0 0]
 [9 9 9]]
```

7.7.8. La función `where`

La función `where` está relacionada con las operaciones condicionales. Cuando se invoca con un único argumento devuelve una tupla con los índices de los elementos del array donde se cumple la condición:

```
>>> a = linspace(1,9,9)
>>> a.shape = (3,3)
>>> print(a)
[[1. 2. 3.]
 [4. 5. 6.]
 [7. 8. 9.]]
>>> d = where (a > 2)
>>> print(d)
(array([0, 1, 1, 1, 2, 2, 2]), array([2, 0, 1, 2, 0, 1, 2]))
```

Cuando `where` se invoca con tres argumentos sustituye los valores que cumplen una condición por un cierto valor y los que no por otros:

```
>>> g = where (a > 2, a**2, -1.0)
>>> print(g)
[[-1. -1.  9.]
 [16. 25. 36.]
 [49. 64. 81.]]
```

Si se necesitan dos condiciones han de usarse los operadores `&` (para 'and') y `|` (para 'or'). Por ejemplo si queremos convertir los valores entre 2 y 6 (excluyéndolos) sería (cuidado con los paréntesis):

```
>>> g = where ((a > 2) & (a < 6), a**2, -1.0)
>>> print(g)
[[-1. -1.  9.]
 [16. 25. -1.]
 [-1. -1. -1.]]
```

7.8. Funciones universales

Estas funciones, denominadas en ocasiones 'puras' y en el argot de Python *ufuncs*, no tienen efectos laterales (no modifican por ejemplo ninguna variable global ni manejan ficheros).

Por ejemplo, `sin` es una *ufunc*. Lo son también la mayoría de las funciones matemáticas que se aplican sobre escalares.

Las *ufuncs* se aplican elemento a elemento.

Las funciones del módulo `math` están redefinidas en `numpy` para que el cálculo sea elemento a elemento de los arrays, y además sea muy rápido:

```
>>> a = array([[3., 2., 4.], [3., 3., 2.]])
>>> c = a+3*sin(a)*log(a)
>>> print(c)
[[ 3.46510853  3.89083084  0.8525469 ]
 [ 3.46510853  3.46510853  3.89083084]]
```

Las *ufuncs* crean nuevos arrays.

Entre las funciones universales se tienen: `absolute`, `add`, `arccos`, `arccosh`, `arcsin`, `arcsinh`, `arctan`, `arctan2`, `arctanh`, `ceil`, `conj`, `cos`, `cosh`, `deg2rad`, `divide`, `exp`, `exp2`, `floor`, `fmod`, `hypot`, `log`, `log10`, `log2`, `mod`, `multiply`, `negative`, `power`, `rad2deg`, `reciprocal`, `remainder`, `rint`, `sign`, `sin`, `sinh`, `sqrt`, `square`, `subtract`, `tan`, `tanh`, `trunc`.

7.8.1. Definición de funciones que tienen como argumentos arrays

En ocasiones necesitamos definir funciones que usen como argumentos arrays. Deben ser funciones puras. Por ejemplo:

```
>>> def f (a):
...     return a+3*sin(a)*log(a)
...
>>> a = array([[3., 2., 4.], [3., 3., 2.]])
>>> f(a)
array([[ 3.46510853,  3.89083084,  0.8525469 ]
       [ 3.46510853,  3.46510853,  3.89083084]])
```

7.8.2. Vectorizando funciones

No obstante, no todas las funciones puras que diseñemos para escalares pueden ser invocadas directamente con arrays.

Ejemplo 7.8.1. *Funciones que no admiten arrays como argumentos.*

Escriba el siguiente código (no olvide la línea comentada):

```
>>> def h (a):  
...     # print("se repite") # <- línea comentada  
...     return a if a > 2 else -a  
...  
>>> a = array([[3., 2., 4.], [3., 3., 2.]])  
>>> h(a)  
ValueError: The truth value of an array ...
```

En el caso anterior se necesita *vectorizar* la función. Ello se hace con un **decorador**, que es otra función que se invoca previamente a la nuestra (y que hace 'cosas' con ella). En este caso se puede poner `@vectorize` en la línea anterior a `def h(a):`.

Ejercicio 7.8.1. Repita el ejemplo anterior utilizando el decorador (`@vectorize`) para que la función `h(a)` admita un array como argumento.

Para vectorizar una función ya definida previamente, se puede usar la función decoradora (`vectorize`) y crear una nueva función vectorizada.

Ejemplo 7.8.2. *Crear una nueva función vectorizada a partir de `h(a)`*

Escriba el siguiente código:

```
>>> hh = vectorize(h) # hh es h 'decorada'  
>>> hh(a)  
array([[ 3., -2., 4.],  
       [ 3., 3., -2.]])
```

Realmente este modo de vectorizar es un «truco», pues Python realiza un bucle, barriendo todos los elementos, como se comprueba descomentando la línea comentada en la definición anterior de `h`.

La vectorización de funciones no es necesaria cuando aparecen operaciones básicas y llamadas a *ufuncs*, como en el caso anteriormente expuesto:

```
def f (a):  
...     return a+3*sin(a)*log(a)
```

Es más, en este caso vectorizar `f` es contraproducente puesto que tarda más la llamada que en el caso presente.

Recomendación: no vectorizar hasta que el intérprete de un error de vectorización, en ese caso vectorizar la función que lo necesite, o rediseñar el código fuente.

Ejercicio 7.8.2. Rediseñe el código del ejemplo 7.8.1 para que admita un array como argumento sin necesidad de recurrir al decorador `@vectorize` ni a la función decoradora `vectorize`.

7.9. Funciones que devuelven valores

Algunas de las funciones que trabajan con arrays y que devuelven valores son:

`all`, `any`, `min`, `max`, `sum`, `prod`, `mean`, `std`, `dot`, `cross`, `outer`, `diag`.

La función `all`

La función `all` realiza un 'y' lógico (`and`) con todos los elementos de un array (o con los elementos a lo largo de un eje). Recordemos que `0` (y `0.0`) se toma como valor `False` y otro valor como `True`.

Escriba el siguiente código para crear un array 3D e ilustrar el comportamiento de la función al seleccionar diferentes ejes:

```
>>> a = arange(24)
>>> a.shape = (2,3,4)
>>> print(a)
[[[ 0  1  2  3]
  [ 4  5  6  7]
  [ 8  9 10 11]]

 [[12 13 14 15]
  [16 17 18 19]
  [20 21 22 23]]]
```

En este array de 3 dimensiones podemos considerar que en la primera dimensión (`axis=0`) tenemos 2 'hojas', en la segunda (`axis=1`) 3 'filas' y en la tercera (`axis=2`) 4 'columnas'.

```
>>> print(a.all())
False
```

La sentencia anterior chequea cada elemento del array y si encuentra algún elemento igual a cero, devuelve `False`.

Llamamos a la función seleccionando ejes:

```
>>> print(a.all(axis=0))
[[False True True True]
 [ True True True True]
 [ True True True True]]
```

El eje 0 representa la primera dimensión de la tupla `shape` que en este caso es (2,3,4), por lo tanto devuelve un array 2D de la forma (3,4) donde ha chequeado los elementos de cada 'hoja' devolviendo `False` en aquella posición donde ha encontrado un cero.

```
>>> print(a.all(axis=1))
```

```
[[False True True True]
 [ True True True True]]
```

El eje 1 es la segunda dimensión de la tupla `shape` que en este ejemplo representa las 'filas'. La función chequea los elementos de cada fila para cada 'hoja' y devuelve por lo tanto un array 2D de la forma (2,4), el 2 debido a que hay 2 'hojas' que analiza por separado y el 4 debido a que cada 'fila' tiene 4 elementos.

```
>>> print(a.all(axis=2))
[[False True True]
 [ True True True]]
```

El eje 2 es la tercera dimensión de la tupla `shape` que representa las 'columnas'. La función chequea los elementos de cada columna para cada 'hoja' y devuelve por lo tanto un array 2D de la forma (2,3), el 2 debido a que hay 2 'hojas' que analiza por separado y el 3 debido a que cada 'columna' tiene 3 elementos.

La función `any`

La función `any` realiza un 'o' lógico (`or`) con todos los elementos de un array (o con los elementos a lo largo de un eje). Recordemos que 0 (y 0.0) se toma como valor `False` y otro valor como `True`.

Utilizaremos el array `a` creado en el ejemplo anterior para ilustrar el comportamiento de la función `any`:

```
>>> print(a.any())
True
>>> print(a.any(axis=0))
[[True True True True]
 [True True True True]
 [True True True True]]
>>> print(a.any(axis=1))
[[True True True True]
 [True True True True]]
>>> print(a.any(axis=2))
[[True True True]
 [True True True]]
```

Observe la respuesta y compare con el caso anterior.

Las funciones `max` y `min`

La función `a.min()` (OJO `min(a)` no funciona) nos devuelve el mínimo valor del array. `a.max()` (OJO `max(a)` no funciona) nos devuelve el máximo valor del array.

Seguiremos utilizando el array `a` creado para los ejemplos anteriores:

```
>>> a = a.T
>>> print(a.shape)
```

```
(4, 3, 2)
>>> a[3,2,1] = 0
>>> a[1,0,0] = 100
>>> print(a)
[[[ 0 12]
   [ 4 16]
   [ 8 20]]

  [[ 100 13]
   [ 5 17]
   [ 9 21]]

  [[ 2 14]
   [ 6 18]
   [10 22]]

  [[ 3 15]
   [ 7 19]
   [11 0]]]
>>> a.min()
0
>>> a.max()
100
>>> print(a.min(axis=0))
[[ 0 12]
 [ 4 16]
 [ 8 0]]
>>> print(a.max(axis=0))
[[100 15]
 [ 7 19]
 [ 11 22]]
```

Compruebe los resultados al cambiar los ejes en las funciones anteriores.

Las funciones `sum` y `prod`

Son funciones similares a `min`, pero devuelven la suma (`sum`) y el producto (`prod`) de los elementos del array:

```
>>> a = arange(12)
>>> a.shape = (4,3)
>>> print(a)
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
>>> print(a.sum())
66
>>> print(a.prod())
0
```

```
>>> print(a.sum(axis=0))
[18 22 26]
>>> print(a.prod(axis=1))
[ 0 60 336 990]
```

Las funciones `mean` y `std`

La función `mean` devuelve la media y `std` la desviación estándar de los elementos del array. Utilizando el array `a` del apartado anterior escriba el siguiente código:

```
>>> print(a.mean(axis=1))
[ 1.  4.  7. 10.]
>>> a.std()
3.452052529534663
```

La función `dot`

El producto escalar de dos vectores (arrays 1D) se obtiene mediante la función `dot`:

```
>>> a = array([3., 2., 4., 0.])
>>> b = array([2., 0., -1., 2.])
>>> dot(a, b)
2.0
```

El tamaño de ambos arrays 1D debe ser el mismo.

La función `dot` también se usa para realizar el producto de matrices por un vector (no confundir con el producto elemento a elemento visto antes).

En el ejemplo siguiente `a` es una matriz 2D y `b` es un vector (tratado como vector columna):

```
>>> a = array([[3., 2., 4.],
               [3., 3., 2.]])
>>> b = array([2., 0., -1.])
>>> dot(a, b)
array([ 2.,  4.])
```

Para más dimensiones en el primer factor, si tiene la forma `(na0, na1, ..., nak)` el segundo factor tiene que tener la forma `(nak,)`. En el ejemplo anterior:

```
>>> a.shape
(2, 3)
>>> b.shape
(3,)
```

Para poder multiplicar dos arrays de 2 o más dimensiones, `a` y `b`, como matrices, el número de elementos a lo largo de la última dimensión de `a` debe ser igual al número de elementos de la penúltima dimensión de `b`. O sea si el primer factor es de la forma `(na0, na1, ..., N)` el segundo tiene que ser de la forma `(nb0, nb1, ..., N, nbj)`.

Pongamos un caso con `a.ndim = 3`, `b.ndim = 3`. Si `c = dot(a, b)` con las formas anteriores entonces:

$$c_{i,j,k,m} = \sum_l^N a_{i,j,l} b_{k,l,m}.$$

A partir de la versión 3.5 de `python` y 1.10 de `numpy` se ha introducido otra función para multiplicar arrays como matrices, denominada `matmul`, que es equivalente al operador `@`. Para casos 2D por 2D y 2D por 1D las funciones `dot` y `matmul` (y por tanto el operador `@`) producen los mismos resultados, pero para más dimensiones no (consúltese la documentación de `numpy`).

La función `cross`

El producto vectorial de dos vectores de 2 o 3 elementos se obtiene mediante la función `cross`. El tamaño de ambos arrays debe ser 2 o 3:

```
>>> x = array([1, 2, 3])
>>> y = array([-1, 2, -4])
>>> cross(x, y)
array([-14, 1, 4])
>>> a = array([1, 2])
>>> b = array([3, 4])
>>> cross(a, b)
array(-2)
```

La función `outer`

Dados dos vectores, `a` de longitud `M` y `b` de longitud `N`, se define de la siguiente manera:

$$c_{ik} = a_i b_k, \quad i = 0, \dots, M-1, \quad k = 0, \dots, N-1.$$

Para calcularlo se utiliza la función `outer`:

```
>>> a = array([1, 2, 3])
>>> b = array([-1, 2])
>>> outer(a, b)
array([[ -1,  2],
       [ -2,  4],
       [ -3,  6]])
```

La función `diag`

La función `diag` nos devuelve la diagonal de un array:

```
>>> a = array([[1, 2, 3, 4], [5, 6, 7, 8]])
>>> print(a)
```

```

[[1 2 3 4]
 [5 6 7 8]]
>>> d = diag(a)
>>> d
array([1, 6])
>>> diag(a, k=1) # 1 encima de la principal
array([2, 7])

```

En las versiones de numpy posteriores a 1.8 el array devuelto no se puede modificar (es *read only*). Si queremos modificar sus elementos debemos copiarlo:

```

>>> d[0] = 99
Traceback (most recent call last):
  File <stdin>, line 1, in <module>
ValueError: assignment destination is read-only
>>> c = diag(a).copy()
>>> c[0] = 99 # OK

```

7.10. Ejercicios

Ejercicio 7.10.1. Un proyectil se dispara con una velocidad de 750 m/s. Calcule la distancia d a la que el proyectil alcanza el suelo si el ángulo de lanzamiento θ cambia de 5° a 85° en incrementos de 5° . Utilice operaciones elemento a elemento. Para visualizar los resultados cree una matriz de 17×2 en la cual los elementos de la primera columna sean los ángulos de lanzamiento, y los de la segunda las correspondientes distancias redondeadas al entero más próximo.

Ejercicio 7.10.2. Dos proyectiles, A y B, se disparan en el mismo instante desde el mismo punto. El proyectil A se dispara a una velocidad de 680 m/s con un ángulo de 65° , mientras que el proyectil B se dispara a una velocidad de 780 m/s con un ángulo de 42° . Calcule qué proyectil llega antes a tierra. Seguidamente, tome el tiempo de vuelo t_v de ese proyectil y divídalo en diez incrementos, creando para ello un vector t con 11 elementos igualmente espaciados (el primer elemento será 0 y el último t_v). Calcule la distancia entre los dos proyectiles para cada uno de los 11 valores de t .

Ejercicio 7.10.3. Un tren y un coche se aproximan a un cruce. En el instante $t = 0$, el tren está a 120 m al sur del cruce, viajando hacia el norte a una velocidad de 86 km/h. En el mismo instante, el coche se encuentra a 60 m al oeste del cruce, viajando hacia el este a una velocidad de 45 km/h y una aceleración de 4 m/s^2 . Determine las posiciones del tren y del coche, la distancia entre ellos, así como la velocidad del tren relativa al coche en cada segundo, durante 10 s. Para presentar los resultados cree una matriz de 11×6 en la que se muestren en columnas: el tiempo, la posición del tren, la posición del coche, la distancia entre el tren y el coche, la velocidad del coche y la velocidad del tren relativa al coche.

Ejercicio 7.10.4. Se mide la aceleración de la gravedad g a diferentes alturas h por encima del nivel del mar. Tomando la altura como variable independiente y la aceleración de la gravedad como variable dependiente se tiene la tabla siguiente:

$h(\text{km})$	$g(\text{m/s}^2)$
10	9.76
20	9.74
30	9.70
40	9.69
50	9.66
60	9.62
70	9.59
80	9.55
90	9.54
100	9.51

Busque en la Wikipedia las fórmulas para un ajuste lineal con dos parámetros en función de los datos (x_i, y_i) , $i = 0, 1, \dots, N - 1$ y realice los cálculos usando numpy sin necesidad de bucles. Determine los parámetros a y h_0 de la expresión $g(h) = h_0 + ah$.

Ejercicio 7.10.5. Diseñe una función `media(a, axis)` que haga lo mismo que la función `mean` utilizando la definición de media:

$$\bar{a} = \frac{\sum_j a_j}{n}$$

Haga que `axis` tenga un valor por defecto `None` y tenga presente que también puede tomar valores positivos y negativos. Compare ambas funciones para comprobar que dan los mismos resultados con arrays de prueba de varias dimensiones.

Ejercicio 7.10.6. Diseñe una función `desvtip(a, axis)` que haga lo mismo que la función `std` utilizando la definición de desviación típica (o estándar):

$$\bar{a} = \sqrt{\frac{\sum_j (a_j - \bar{a})^2}{n}}$$

Haga que `axis` tenga un valor por defecto `None` y tenga presente que también puede tomar valores positivos y negativos. Es recomendable el uso de `expand_dims` para facilitar el cálculo. Compare ambas funciones para comprobar que dan los mismos resultados con arrays de prueba de varias dimensiones.

7.11. Interés de vectorizar el código

Eliminar índices del código es posible siempre que las expresiones se puedan evaluar a la vez para cualquier índice (si no es así no se puede vectorizar el código). Además, siempre que se eliminen del código índices que deban ser iterados con un bucle tanto mejor (en cualquier lenguaje interpretado al menos). Pero eliminar índices implica utilizar más memoria y **repensar** el código (lo que a veces es difícil).

Analizaremos varios casos de vectorización de expresiones, algunos con más contenido físico que otros.

Ejemplo 7.11.1. *Un caso simple*

Sea $\mathbf{b}$ un array cuadrado de forma (n, n) . La expresión:

$$c_{ik} = b_{ik} + b_{i+1,k+1}, \quad \begin{cases} i = 0, \dots, n-2, \\ k = 0, \dots, n-2, \end{cases}$$

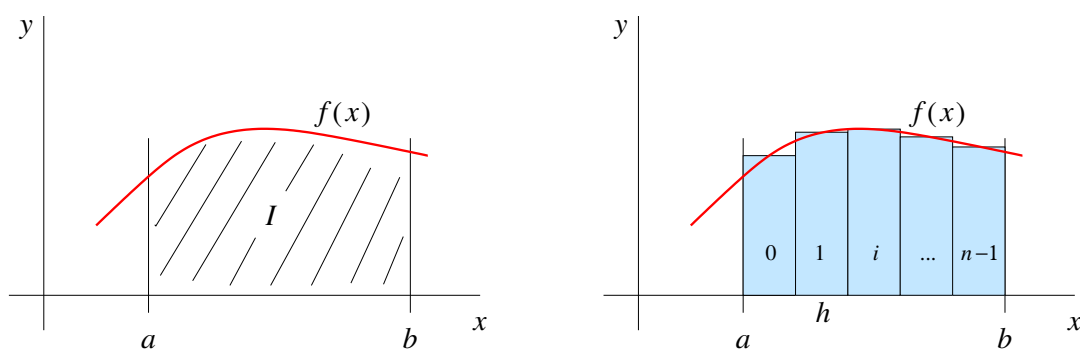
se codifica:

```
c[:-1,:-1] = b[:-1,:-1] + b[1:,1:]
```

Ejemplo 7.11.2. *Integración por rectángulos*

Se define la integral de Riemann de una función $f(x)$ entre dos abscisas a y b mediante:

$$I = \int_a^b f(x) dx.$$



La solución de este problema puede obtenerse numéricamente por el método de los rectángulos, usando como altura de cada rectángulo el valor de la función en la abscisa media de cada rectángulo:

$$I \approx h \sum_{i=0}^{n-1} f\left(a + \frac{1}{2}h + ih\right), \quad h = \frac{b-a}{n}.$$

Se puede usar la siguiente función:

```
def integra (f, a, b, n=100):
    h = (b-a)/float(n)
    x = linspace(a+0.5*h, b-0.5*h, n)
    return h*sum(f(x))
```

La vectorización va implícita en la llamada a la función $\mathbf{f}$ para luego realizar la suma.

Ejemplo 7.11.3. *Ecuación de Poisson en 2D*

La ecuación de Poisson del electromagnetismo es en 2D:

$$\frac{\partial^2 V}{\partial x^2} + \frac{\partial^2 V}{\partial y^2} = -\frac{\rho}{\epsilon_0},$$

siendo V el potencial eléctrico y ρ la densidad de carga en cada punto del recinto.

Discreticemos un recinto rectangular con un mallado h (tanto en Ox como en Oy) con n_x nodos en Ox y n_y nodos en Oy , de tal manera que $V(i, k)$ define el potencial en los puntos $i = 0, 1, \dots, n_x - 1$, $k = 0, 1, \dots, n_y - 1$. El problema de Dirichlet (potencial conocido en la frontera) se puede resolver iterando en m :

$$V_{i,k}^{m+1} = \frac{1}{4} \left(V_{i+1,k}^m + V_{i-1,k}^m + V_{i,k+1}^m + V_{i,k-1}^m + \frac{h^2}{\epsilon_0} \rho_{i,k} \right)$$

$$i = 1, 2, \dots, n-2, k = 1, 2, \dots, n-2.$$

Denominando K a h^2/ϵ_0 , y R a ρ tendríamos:

$$Vn[1:-1, 1:-1] = (V[2:, 1:-1] + V[:, -2, 1:-1] + V[1:-1, 2:] + V[1:-1, :-2] + K * R[1:-1, 1:-1]) / 4$$

El potencial en los bordes no se calcula en esta expresión (no varía con el tiempo). Para el siguiente paso temporal se hace:

$$V[1:-1, 1:-1] = \text{copy}(Vn[1:-1, 1:-1])$$

Ejemplo 7.11.4. Difusión de neutrones en 1D

En un reactor nuclear de fisión (en 1D para simplificar) si denominamos y a la cantidad de neutrones presentes, C a su tasa de creación y D al coeficiente de difusión, se verifica la siguiente ecuación diferencial en derivadas parciales:

$$\frac{\partial y}{\partial t} = D \frac{\partial^2 y}{\partial x^2} + Cy.$$

Se supone que $y(x=0) = y(x=L) = 0$, siendo $x=0$ y $x=L$ los bordes del recinto.

Esta ecuación se puede discretizar en el espacio usando n intervalos, de tal manera que $h = L/n$, y en el tiempo usando un incremento temporal τ . Con $i = 1, 2, \dots, n-2$ para los puntos en el espacio, e indicando m el instante temporal, tendríamos:

$$y_i^{m+1} = y_i^m + \frac{D\tau}{h^2} (y_{i+1}^m + y_{i-1}^m - 2y_i^m) + C\tau y_i^m.$$

Este método permite estudiar el problema de la criticidad (automantenimiento de la reacción nuclear) variando D , C y la condición inicial, y_i^0 .

Denominando K a $D\tau/h^2$, y R a $1 + C\tau - 2K$ tendríamos:

$$yn[1:-1] = R * y[1:-1] + K * (y[2:] + y[:-2])$$

Para el siguiente paso temporal se hace:

$$y[1:-1] = \text{copy}(yn[1:-1])$$

Ejemplo 7.11.5. Serie radiactiva (opcional)

Las series radiactivas sin bifurcaciones están formadas por radionúclidos que se desintegran unos en otros: $0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow n$, siendo el núclido n estable. Si N_i es el número de núcleos presentes del radionúclido i se tiene:

$$\begin{aligned}\dot{N}_0 &= -\lambda_0 N_0, \\ \dot{N}_i &= \lambda_{i-1} N_{i-1} - \lambda_i N_i.\end{aligned}$$

λ_i es la denominada constante de desintegración del radionúclido i .

Se puede poner en forma matricial:

$$\dot{\mathbf{N}} = \begin{pmatrix} -\lambda_0 & 0 & 0 & \dots & 0 \\ \lambda_0 & -\lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_1 & -\lambda_2 & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & \dots & -\lambda_{n-1} \end{pmatrix} \mathbf{N}.$$

Si denominamos $\mathbf{L}$ al vector con las λ :

$\mathbf{L} = [\mathbf{L}_0, \mathbf{L}_1, \dots]$

y $\mathbf{N}$ al vector de número de núcleos:

$\mathbf{N} = [\mathbf{N}_0, \mathbf{N}_1, \dots]$

se vectoriza de la siguiente manera:

```
n = N.size # n = L.size, también
Np = dot((eye(n, n, k=-1)-eye(n, n))*L, N)
```

Al operar con $\mathbf{L}$ y $\mathbf{N}$ se realiza broadcasting.

Ejemplo 7.11.6. Eliminación sucesiva de índices (opcional)

(La idea de este ejemplo está tomada de Numerical Recipes in Fortran 90.)

Sea $\mathbf{x}$ un vector de longitud n . La expresión:

$$w_i = \sum_{k=0}^{n-1} |x_i + x_k|, \quad i = 0, \dots, n-1,$$

se puede codificar sin vectorizar:

```
w = zeros_like(x)
indices = range(len(x))
for i in indices:
    w[i] = sum(abs(x[i]+x))
```

pero también mediante:

```
n = len(x)
x.shape = (n, 1)
aux = x.repeat(n, axis=1)
w = sum(abs(aux+aux.T), axis=0)
```

Este segundo método (usado por ejemplo en las versiones modernas de Fortran, aunque obviamente el código no es éste) necesita mucha más memoria que el primero (el array bidimensional `aux` al menos), pero es más rápido, ya que no lleva bucles.

En Python con `numpy` se puede todavía mejorar puesto que el array `aux` no hace falta explícitamente (sí implícitamente) ya que el broadcasting nos permite poner:

```
w = sum(abs(x.reshape(-1,1)+x), axis=0)
```

Ejemplo 7.11.7. Tratamiento de un caso con excepciones (opcional)

(La idea de este ejemplo está tomada de Numerical Recipes in Fortran 90.)

Sea `x` un vector de longitud `n`. La expresión:

$$w_i = \prod_{k=0, k \neq i}^{n-1} (x_k - x_i), \quad i = 0, \dots, n-1,$$

(la excepción es que $k \neq i$) se puede codificar:

```
w = zeros_like(x)
indices = range(len(x))
for i in indices:
    w[i] = prod(where(x != x[i], x-x[i], 1.0))
```

pero es más rápido el trozo de código siguiente:

```
n = len(x)
x.shape = (n, 1)
aux = x.repeat(n, axis=1)
aux = aux-aux.T
w = prod(where(aux != 0, aux, 1.0), axis=0)
```

Como vemos, la eliminación de un índice 'que se pierde' (`k` cuando es igual al `i`) conlleva la utilización de la función `where`, como máscara.

De nuevo el broadcasting de `numpy` nos permite mejorar el código:

```
aux = x.reshape(-1,1)-x
w = prod(where(aux != 0, aux, 1.0), axis=0)
```

Conviene en esta ocasión mantener el array `aux` ya que se utiliza dos veces en el producto.

Ejemplo 7.11.8. Movimiento planetario (opcional)

El ejemplo anterior puede servir de partida, por ejemplo, para codificar el cálculo de la aceleración de los planetas y el Sol en una simulación del sistema solar, ya que también hay un índice 'que se pierde'. También valdría para simular el movimiento de un cúmulo de estrellas (que tiene cientos y miles de estrellas en interacción).

Suponiendo los n cuerpos puntuales en el espacio, se tienen unas componentes de la aceleración del cuerpo i :

$$a_i^c = - \sum_{k=0, k \neq i}^{n-1} \frac{Gm_k(x_i^c - x_k^c)}{(r_{ik})^{1.5}}, \quad c = 0, 1, 2 \quad i = 0, 1, \dots, n-1.$$

siendo $r_{ik} = \sum_c (x_i^c - x_k^c)^2$, la distancia al cuadrado entre el cuerpo i y el k .

Suponiendo un vector $\mathbf{Gm}$ con las masas multiplicadas por la G de la gravedad, y un vector de posiciones $\mathbf{x}[\mathbf{nc}, \mathbf{n}]$, siendo $\mathbf{nc}$ el número de dimensiones (2 ó 3 dependiendo del problema), el código sin vectorizar sería:

```
def acel_no_vec (x, m):
    nc, n = x.shape
    a = zeros_like(x)
    for i in range(n):
        for k in range(n):
            if i != k:
                rik = sum((x[:,i]-x[:,k])**2)**1.5
                for c in range(nc):
                    a[c,i] -= Gm[k]*(x[c,i]-x[c,k])/rik
    return a
```

Realmente si se vectoriza una suma.

Se puede vectorizar con `where` para eliminar los casos en los que $i = k$, y tendríamos:

```
def accelvec_con_where (x, m):
    nc, n = x.shape
    d = reshape(x, (nc, n, 1)).repeat(n, axis=2)
    d = d-transpose(d, axes=(0, 2, 1))
    r3 = sum(d**2, axis=0)
    r3 = where(r3 != 0, r3**-1.5, 0.0)
    return -sum(m*d*r3, axis=2)
```

Se ve que han desaparecido los bucles. Esta versión es típicamente 140 veces más rápida que la no vectorizada, para $\mathbf{nc}=3$, $\mathbf{n}=1000$. (Nota: en la instrucción donde está `where` el intérprete avisa con «RuntimeWarning: divide by zero encountered in power», pero puede obviarse el aviso.)

Otra mejora se obtiene no usando `where` sino haciendo igualmente el cálculo, aunque dé infinita la inversa de la distancia cuando $i = k$ (o sea en la diagonal principal), pero poniendo a cero tales valores con `ravel`, de la manera siguiente:

```
def accelvec_con_ravel (x, m):
    nc, n = x.shape
    d = reshape(x, (nc, n, 1)).repeat(n, axis=2)
    d = d-transpose(d, axes=(0, 2, 1))
    r3 = sum(d**2, axis=0)**-1.5
    r3.ravel()[0::n+1] = 0.0
    return -sum(m*d*r3, axis=2)
```

Esta versión es típicamente 150 veces más rápida que la no vectorizada, para $\mathbf{nc}=3$, $\mathbf{n}=1000$. (Nota: en la instrucción donde se calcula `r3` el intérprete avisa con «RuntimeWarning: divide by zero encountered in power», pero puede obviarse el aviso.)

Un paso más en la vectorización es usar el broadcasting para eliminar `repeat` y `transpose`, teniéndose finalmente:

```
def acelvec_con_ravel_broad (x, m):  
    nc, n = x.shape  
    d = x.reshape((nc, n, 1))-x.reshape((nc, 1, n))  
    r3 = sum(d**2, axis=0)**-1.5  
    r3.ravel()[0:n+1] = 0  
    return -sum(m*d*r3, axis=2)
```

Esta versión es típicamente 200 veces más rápida que la no vectorizada, para `nc=3`, `n=1000`. (Nota: en la instrucción donde se calcula `r3` el intérprete avisa con «RuntimeWarning: divide by zero encountered in power», pero puede obviarse el aviso.)

Para el sistema solar, para el que hay 10 cuerpos (tomando sólo los planetas y el Sol), la velocidad se multiplica por entre unas 5 y 10 veces al vectorizar.

Los números de velocidad de cálculo expuestos son sólo aproximados, y dependen mucho del ordenador y de cada caso concreto.

Como vemos en estos ejemplos, merece la pena repensar el código para vectorizar en lo posible el código con `numpy`.